

How to use Mixture in your projects

Mixture is a deliberately-small, layered skills + agents framework for **Claude Code**. This guide shows how to drop it into another project and use each layer. Nothing here is theoretical — every command below is in this repo and tested.

Prerequisites: Node.js $\geq$ 18, Claude Code. No npm dependencies — the tooling is pure Node built-ins.

TL;DR — fastest path

```
# 1. Make the skills available to Claude Code globally (symlink each skill dir)
for d in /path/to/mixture/skills/*/; do
  [ -f "$d/SKILL.md" ] && ln -sf "$d" "$HOME/.claude/skills/$(basename "$d")"
done

# 2. (optional) copy the L3/L4 tooling into your project if you want hooks/coordination
cp -r /path/to/mixture/{scripts,hooks,manifests,coordination,references} your-project/

# 3. Verify everything is wired
cd your-project && node /path/to/mixture/scripts/validate-frontmatter.mjs
```

That's enough to get the **skills** (L1–L2). L3 (hooks/memory) and L4 (coordination) are opt-in — see below.

The four layers (what you're getting)

Layer	What it is	Where
L1 Behavioral kernel	One always-relevant coding-discipline skill	skills/kernel/coding-behavior
L2 Skills + authoring gate	grill-me, diagnose, tdd, context, code-review + write-a-skill	skills/engineering/* , skills/meta/*
L3 Governance	Install profiles, env-governed hooks, session memory, CI gate	manifests/ , hooks/ , scripts/
L4 Coordination	Multi-agent task ledger + heartbeat runtime	coordination/ , skills/coordination/*

Lower layers never depend on higher ones. Use only the layers you want.

Installing the skills (L1–L2)

A "skill" is a directory with a `SKILL.md`. Claude Code auto-loads a skill when your request matches its `description`. Three ways to install:

A. Global symlink (all your projects get them): the TL;DR loop above, linking into `~/ .claude/skills/`.

B. Per-project: copy the skill dirs you want into `your-project/.claude/skills/`:

```
cp -r /path/to/mixture/skills/engineering/diagnose your-project/.claude/skills/
```

C. As a plugin: the repo ships `.claude-plugin/plugin.json` listing all skills, so it can be installed as a Claude Code plugin (`/plugin`) if you host it in a marketplace.

Pick a subset with profiles (`manifests/install-profiles.json`): `minimal` (kernel only), `dev` (kernel + engineering), `authoring`, `coordination`, `full`. Install only the skills a profile lists to keep your context budget small.

Using the skills

Just work normally — Claude routes to a skill by its `description`. Or invoke explicitly by name: - `coding-behavior` (kernel) — fires on real coding tasks; biases toward asking before assuming, minimal diffs, surgical changes, and test-verifiable goals. - `grill-me` — before a big build, interrogates requirements one question at a time. - `diagnose` — for bugs: builds a deterministic repro *first*, then tests ranked hypotheses. - `tdd` — vertical-slice red-green-refactor. - `context` — maintains a `CONTEXT.md` domain glossary (shared human↔agent vocabulary). - `code-review` — prioritized correctness/security findings with `file:line`, not style nits.

Governance (L3) — optional but recommended

Install profiles

`manifests/install-profiles.json` maps a profile name → a set of skill paths. Use it to decide what to install per project (a heavy repo can run `minimal`).

Hooks, governed by env (never by editing files)

`hooks/hooks.json` is the annotated source of truth. Resolve it into a Claude Code-consumable config:

```
node scripts/resolve-hooks.mjs          # prints the active hook config for your
profile
node scripts/resolve-hooks.mjs --explain # shows which hooks are on/off and why
```

Control behavior with env vars — no file edits: - `MIXTURE_HOOK_PROFILE=off|standard|strict` (default `standard`) - `MIXTURE_DISABLED_HOOKS=load-memory,save-memory` (comma-separated hook names)

To wire into Claude Code, put the resolved output's `hooks` into your project `.claude/settings.json`.

Session memory (survives compaction)

Durable facts live outside the context window in `.mixture/memory/store.json`:

```
# add a durable fact (or wire as a tool the agent calls)
node hooks/memory-persistence/save.mjs --add "Chose Postgres over SQLite for multi-
writer support" --type decision --pin
node hooks/memory-persistence/load.mjs      # SessionStart: prints a digest for context
injection
node hooks/memory-persistence/save.mjs      # PreCompact: consolidate + dedupe
(idempotent)
node hooks/memory-persistence/clean.mjs     # SessionEnd: enforce TTL + max (never evicts
pinned)
```

Types: `goal | decision | constraint | glossary | fact`. Tunables:
`MIXTURE_MEMORY_{DIR,LOAD_LIMIT,MAX,TTL_DAYS}`.

The CI gate (keep the catalog honest)

```
npm run ci # = validate-frontmatter + check-drift --ci + resolve-hooks --check +
coordination tests
```

- `validate-frontmatter.mjs` — every `SKILL.md` has a valid routing contract (kebab name, ≤ 1024 -char description containing "Use when").
- `check-drift.mjs` — every shipped skill is registered + in a profile + has an eval, and the catalog is under the ~30 cap. **In-editor (advisory) it warns; with `--ci` it's fatal.** (See ADR-0002.)

Copy `.github/workflows/ci.yml` to gate this on every PR.

Coordination (L4) — multi-agent runtime, optional

Use this only if you actually run multiple agents that share work. It's a task ledger with every safety invariant enforced in code.

Task ledger CLI

```
export MIXTURE_COORD_DIR=.mixture/coordination # where the ledger lives (gitignore it)

node coordination/cli.mjs create --title "Design schema" --priority 1
node coordination/cli.mjs create --title "Build API" --priority 1
node coordination/cli.mjs block --id T-2 --by T-1 # T-2 waits on T-1
node coordination/cli.mjs tick --agent worker-1 --run r1 # atomically claim top-
priority ready work
node coordination/cli.mjs done --id T-1 # auto-resumes T-2
node coordination/cli.mjs ask --id T-2 --kind request_confirmation --prompt "Ship?" --
key ship:T2:r1
node coordination/cli.mjs resolve --iid I-1 --response yes # wakes the assignee
```

Exit codes (machine-detectable — the agent never parses prose)

0 ok · 2 usage · 3 lock-busy · 5 stale-gate · 6 cycle · 7 budget-exhausted · 8 blocked · 9
CONFLICT → another agent owns it; **STOP**, never retry.

The behavioral contract

Install the `coordination-protocol` skill — it tells a woken agent exactly how to behave (identity → pick work → atomic checkout → do → delegate → human-gate), and to obey the exit codes. Hard invariants live in `coordination/ledger.mjs`; the skill is the behavior.

Run a heartbeat (Claude Code native)

- **Scheduled:** create a cron routine that fires every N minutes and runs `cli.mjs tick`, then executes any assigned task. (We use a recurring job at `:17,:47`.)
- **Self-paced:** an agent loops via `ScheduleWakeup` after each tick.
- **Dispatch many:** fan out one subagent per ready task with the `Agent` tool — atomic checkout guarantees no two touch the same task. (Proven in `coordination/DEMO.md`: a 4-agent contention test produced exactly one winner, no double-work.)

Extending: add your own skill (the right way)

Always go through the gate skill `write-a-skill`. The rules it enforces: 1. **description** = routing **contract**: ≤1024 chars, sentence 1 = what it does, sentence 2 = "Use when ...". 2. **Progressive disclosure**: `SKILL.md` ≤ ~100 lines; deeper detail in `references/*.md` one level deep. 3. **Earn your context** — prevents a specific failure the kernel/existing skills don't. Catalog cap ~30. 4. **Examples-as-docs**: at least one / pair. 5. **Mandatory eval**: add `evals/<skill-name>.eval.md`. 6. **Register**: add to `.claude-plugin/plugin.json` and ≥1 profile.

Then: `node scripts/validate-frontmatter.mjs && node scripts/check-drift.mjs --ci` must pass.

Authoring tip: write the `SKILL.md` first, then register, then add the eval — the in-editor drift hook is *advisory* so it won't block this order; CI enforces completeness at merge (ADR-0002).

Environment variables reference

Var	Default	Purpose
<code>MIXTURE_HOOK_PROFILE</code>	<code>standard</code>	<code>off</code> / <code>standard</code> / <code>strict</code> — which hooks are active
<code>MIXTURE_DISABLED_HOOKS</code>	(none)	comma-separated hook names to disable
<code>MIXTURE_MEMORY_DIR</code>	<code>.mixture/memory</code>	memory store location
<code>MIXTURE_MEMORY_LOAD_LIMIT</code>	<code>50</code>	max entries injected at SessionStart
<code>MIXTURE_MEMORY_MAX</code> / <code>MIXTURE_MEMORY_TTL_DAYS</code>	<code>200</code> / <code>90</code>	memory bound
<code>MIXTURE_COORD_DIR</code>	<code>.mixture/coordination</code>	task ledger location
<code>MIXTURE_DRIFT_STRICT=1</code>	—	make <code>check-drift</code> fatal (same as <code>--ci</code>)

Keep the philosophy (or it rots)

Mixture's value is its discipline, not its size. If you adopt it, keep the guardrails: - **The ~30-skill cap.** Adding one may mean deprecating one. Volume is the failure mode (see `docs/premortem.md`). - **Every skill has an eval.** No eval = a guess; CI rejects it. - **Enforce in code, not prose.** Invariants belong in `scripts/ / ledger.mjs`, not just instructions. - **Don't build L4 before you need it.** It's powerful and risky; use the manual ledger until a real fleet exists.

See `docs/architecture.md` (what each idea was taken from), `docs/premortem.md` (10 ways it fails + guards), and `docs/adr/` (decisions) for the full reasoning.